

PRACTICAL 1 AND 2

Initializing Repository

Check the git version

```
$ git --version
```

Set the user name and email id (skip --global for local directory)

```
$ git config user.name "----"
```

```
$ git config user.email "----"
```

List all attributes and their corresponding values

```
$ git config --list
```

```
$ git config user.name
```

Change the directory (locate your project folder)

Initialize the git repository and check the status

```
$ git init
```

```
$ git status
```

After adding a file in the folder Git Project again check the status

```
$ git status (it will show saved file)
```

Add the file for tracking and then check the status. It is ready to be committed

```
$ git add README.txt
```

```
$ git status
```

Added few more files in the folder

```
$ git status
```

```
$ git add --a
```

```
$ git status
```

Execute the commit command

```
$ git commit -m 'initial project version'
```

Staging

Edit any one of the tracked files and check the status again.

Add the file in staging and check the status, the file is ready to be committed.

\$ git status

\$ git add "README.txt"

\$ git status

Before tracking, step to ignore specific types of files:

Create a file in your project folder named as .gitignore and open it with notepad

Now onwards if any new .txt file will be added in the project folder the file will be ignored by

git tracking

\$ git log -p

Viewing the Commit History

Check all the previous commit

\$ git log

Check some abbreviated stats for each commit

\$ git log -p

\$ git log --stat

Check the commit log on online

\$ git log --pretty=oneline

Format the commit log

\$ git log --pretty=format:"%h- %an,%ar:%s"

Show the list of files affected with added/modified/deleted information as well

\$ git log --name-status

Limiting Log Output

```
$ git log --since=2.weeks
```

```
$ git log -n 1
```

```
$ git log -1
```

```
$ git log --after="today 7 pm"
```

```
$ git log --before="2026-12-10 22:00:00"
```

```
$ git log --author="name" --since="7 days ago"
```

Limiting Log Output according to Date/Time

```
$ git log --after="2026-2-10 11:00:00" --before="2025-12-10 14:00:00" --  
pretty=fuller
```

Using a GUI to Visualize History

Open CMD from project folder

Gitk

Undoing Things

Changing Your Last Commit

Changing commit msg

```
$ git commit -a -m "Commit for --amend"
```

```
$ git commit --amend
```

A notepad file will open, edit the commit message, save the changes and close it. Then check

the log again

```
$ git log
```

Unstaging a Staged File

```
$ git add
```

```
$ git status
```

```
$ git reset HEAD README.txt
```

```
$ git status
```

Or

```
$ git status
```

```
$ git restore --staged README.txt
```

```
$ git status
```

Unmodifying a Modified File

```
$ git status
```

```
$ git restore --staged README.txt
```

```
$ git status
```

```
$ git restore README.txt
```

```
$ git status
```

Working with Remotes

Cloning remote repository

```
$ cd [folder path]
```

```
$ git clone https://github.com/schacon/ticgit
```

Showing Your Remotes

Changing directory

```
$ cd ticgit
```

```
$ git remote
```

Specify -v, which shows you the URL that Git has stored for the short name to be expanded

to:

```
$ git remote -v
```

Adding Remote Repositories

```
$ git remote -v
```

Fetching and Pulling from Your Remotes

```
$ git fetch pb
```

```
$ git push origin master
```

Inspecting a Remote

```
$ git remote show origin
```

Removing and Renaming Remotes

```
$ git remote
```

```
$ git remote rename pb paul
```

```
$ git remote
```

```
$ git remote rm paul
```

```
$ git remote
```

Tagging

Listing Your Tags

\$ git tag

Creating Tags

Annotated Tags

\$ git tag -a v0.0 -m 'myversion 0.0'

\$ git tag

Signed Tags :- Use -s instead -a

\$ git tag -s v0.1 -m 'signed version 0.1'

Lightweight Tags :- Don't use -a, -s and -m

\$ git tag v0.2

Verifying Tags

\$ git tag -v v0.0

Tagging Later

\$ git tag -a v0.0 0d82a2e -m "version 0.1"

Sharing Tags

\$ git push origin v0.0

Git Aliases

\$ git config --global alias.ci commit

\$ git config --global--get-regexp alias

PRACTICAL 3

Creating a New Branch

\$ git branch test

Switch between Branch

\$ git checkout test

Shorthand for create and switch

\$ git checkout -b test

Make some changes in any file

\$ git status

\$ git commit -a -m 'Updated index.html'

\$ git status

Now switch back to master branch

\$ git checkout master

Note: Check the file you have updated and notice the changes

Again make some changes and commit

\$ git status

\$ git commit -a -m 'Updated index.html from master branch'

\$ git status

Create one more branch and switch

Make some changes in the file and commit

\$ git checkout -b hotfix

\$ git commit -a -m 'Updated index.html from hotfix'

\$ git status

Merge hotfix with master

\$ git checkout master

\$ git merge hotfix

List all the branches

\$ git branch

Delete branch

\$ git branch -d hotfix

\$ git branch

Conflict while merging

\$ git merge test

Open the file and check the conflict

Keep one of the changes, save and close it

Commit the changes it will get merged now

Branch Management

To check last commit on each branch

\$ git branch -v

To display merged and not merged branches

\$ git branch --merged

\$ git branch --no-merged

PRACTICAL 4

Repository Creation & Version Control

To create and manage a GitHub repository using basic Git commands.

1. Open **<https://github.com>** and log in.
2. Click New Repository.
3. Enter **repository name** (without any spaces)
4. Select Public → Click Create Repository.
5. Open **Git Bash** on local machine.
6. Navigate to your project folder using cd command.
7. Then run git init command.

\$ cd [folder path]

\$ git init

8. Add and/or update some content in that folder.
9. Add the changes in staging state using following command:

\$ git add .

10. Then commit the changes using following command:

\$ git commit -m "Initial Commit"

11. Link GitHub repository using following command:

\$ git remote add origin <repo-url>

12. Push code using:

\$ git push -u origin main

13. Check your project files in GitHub repository. It should have all the changes you have pushed now.

14. Modify any file from GitHub account and commit the changes.

Open any file → click edit → make changes → click commit changes

15. Pull the changes on local machines.

\$ git pull origin main

PRACTICAL 5

Branching, Merging & Conflict Resolution

To create branches, merge them, and resolve conflicts.

Part A: Create Branch on GitHub

1. Open your **repository on GitHub**.
2. Click the **branch dropdown** (usually shows main).
3. Type branch name: **feature-branch**.
4. Click Create branch: **feature-branch**.

Part B: Modify File in Feature Branch

5. Ensure **feature-branch** is selected.
6. Open any **text file**.
7. Click **Edit**.
8. Add content like: "**This change is from feature branch.**"
9. Scroll down → **Add commit message**.
10. Click **Commit changes** (commit directly to feature-branch).

Part C: Modify Same File in Main Branch

11. Switch back to **main branch**.
12. Open the **same file**.
13. Click **Edit**.
14. Add different content like: "**This change is from main branch.**"
15. **Commit changes** to main branch.

Part D: Merge Branch and Create Conflict

16. Go to Pull Requests → **New Pull Request**.
17. Base: **main**, Compare: **feature-branch**.
18. Click **Create Pull Request**.
19. GitHub will show **merge conflict warning**.

Part E: Resolve Conflict on GitHub

20. Click **Resolve conflicts**.

21. GitHub shows conflict markers:

```
<<<<<< main
```

Main branch content

```
=====
```

Feature branch content

```
>>>>>> feature-branch
```

22. **Edit** file to keep required content.

23. **Remove** conflict markers.

24. Click **Mark as resolved**.

25. Click **Commit merge**.

PRACTICAL 6

GitHub Collaboration using Pull Requests

To collaborate using fork, pull request, and review mechanism.

1. Open **someone else's repository**.
2. Click **Fork**.
3. Clone forked repository locally.
4. Create a new branch.

```
$ cd [folder path]
```

```
$ git clone [repo url]
```

```
$ git checkout -b collaboration-branch
```

5. Make changes and commit.

```
$ git add .
```

```
$ git commit -m "Added changes for collaboration Pract4"
```

6. Push changes to GitHub.

```
$ git push origin collaboration-branch
```

7. Click **New Pull Request**.
8. Add **description** and submit **Pull Request**.
9. **Review** and **merge** Pull Request.

PRACTICAL 7

Raise issue in github

1. Login to **GitHub**
2. Go to **<https://github.com>**
3. Sign in with your account.
4. Open the **Repository**
5. Go to the repository where you want to **raise the issue**.
6. Click on **Issues Tab**
7. Click on **new issue** button
8. If you have some group member you can assign issue to your group members
9. Set the **label** as well if required
10. If you have **public** repository people can comment and suggest some solution for the issue
11. Once it is resolved we can **close the issue**
12. If someone wants to change the code then can propose changes
13. Then go to **code** tab
14. Click on **compare and pull request** and scroll down you will find the **comparison**
15. Then if you want to **merge** those changes suggested in your project then click on **Create Pull**
16. **Request** button
17. Then click on **merge pull request**
18. Click on **Confirm merge**

PRACTICAL 8

Installation and Project Set Up

1. Install **JDK** on Windows 11
 - a. Set **system variables**
 - b. Press **Windows** Key
 - c. Search → **Environment Variables**
 - d. Click → **Edit the system environment variables**
 - e. Click → **Environment Variables**
 - f. Under System Variables → **Click New**
 - g. Add: Variable Name: **JAVA_HOME** and Variable Value: **C:\Program Files\Java\jdk-17**
 - h. (Adjust path as per your installation)
 - i. Add **Java to PATH**
 - j. Select **Path**
 - k. Click **Edit**
 - l. Click **New**
 - m. Add: **%JAVA_HOME%\bin**
 - n. Click **OK**
 - o. Verify **Java**
 - p. Open Command Prompt (cmd):
 - i. **java -version**
 - ii. **javac -version**
 - q. You should see version details.

2. Install **Maven** on Windows 11

3. Download Maven from: <https://maven.apache.org/download.cgi>

Choose: Binary zip archive

4. Extract Maven

5. Inside it should contain: bin, conf, lib

6. Set system variables for maven as well

a. Go to **Environment Variables** again

b. Click **New**

c. Add: Variable Name: **MAVEN_HOME** and Variable Value :
C:\apache-maven

d. Add **Maven to PATH**

e. Edit Path, Add: **%MAVEN_HOME%\bin**

f. Click **OK**.

g. Verify **Maven**

h. Open new Command Prompt:

i. **mvn -version**

j. You should see: **Maven version, Java version, OS: Windows 11**

7. Create a folder for your project and open cmd from this folder,
Enter following, command to start creating structure of your project

a. **mvn archetype:generate**

b. When prompted:

c. Choose: **maven-archetype-quickstart or default**

d. Choose: **version or set it to latest**

e. Enter:

- i. GroupId: **com.college**
- ii. ArtifactId: **MavenDemo**
- iii. Version: **1.0-SNAPSHOT**
- iv. Package: **com.college**

8. Project created successfully.

9. System will ask for confirmation press: **Y**

10. Open your folder inside this MavenDemo Project is created

11. This folder contains **src** and **pom.xml** file

12. Src folder contains main and test folders

13. In \main\java\org\myapp\com\ App.java file is created you can open and check the same

14. Switch back to **cmd** and run Maven Commands

- a. **mvn compile**

- b. **mvn test**

- c. **mvn package**

15. Check: **target folder**

16. You will see: **MavenDemo-1.0-SNAPSHOT.jar**

17. Now from your Maven project folder open cmd and enter following command to execute the project

**\$ java -cp target/MavenDemoApp-1.0-SNAPSHOT.jar
org.myapp.com.App**

PRACTICAL 9

Creating web application with maven

1. Open cmd from your folder where you want to save your web app
2. Then enter following command
 - a. **mvn archetype:generate**
 - b. Note: Copy the list of archetypes and paste in notepad then search for **webapp-j2ee** and note the id
3. Select archetype ; - **2655:remote**
4. Select the version and enter details as below:
5. GroupId: **org.mywebapp.com**
 - i. ii. ArtifactId: **MyWebApp**
 - ii. Version: **1.0-SNAPSHOT**
 - iii. Package: **enter**
6. In your chosen folder you will find a folder **MyWebApp**
7. Enter following command to change the directory
 - a. **cd MyWebApp**
8. Compile and package the default web app with following commands
 - a. **mvn compile**
 - b. **mvn package**
9. Now in MyWebApp folder you will find a target folder and in target folder **MyWebApp-1.0-SNAPSHOT.war** file
10. Now come back to MyWebApp and open src folder
 - a. Go to \main\webapp you will see default **index.jsp** file
 - b. Now we will deploy our war file with tomcat server
 - c. Open \apache-tomcat-9.0.106\bin\ and open **startup.bat** file then open browser and type :- **localhost:8080**
 - d. Click on **Manage App**

- e. It will ask you to enter username and password, to check the same go to \apache-tomcat-9.0.106\conf\ and open tomcat-users.xml file with notepad check the username and set the password for the same also add user role and remove the comment tag

f. Replace the above code with following:

```
<role rolename="manager-gui"/>
<user username="admin" password="12345" roles="manager-gui"/>
```

11.

Now stop and then restart tomcat server go to browser and refresh **localhost:8080** and click on **Manage App**

12. Enter **username** and **password** it will open

13. Scroll Down

14. Click on **choose file button**:

15. Go to \myapp\MyWebApp\target\ and select **.war** file and click on **deploy button**

16. After this you can find your .war file in the above list

17. Just click on it and you will see the output:

a. localhost:8080/MyWebApp-1.0-SNAPSHOT/

PRACTICAL 10

Introduction to maven plugins

A. Using maven compiler plugin

Open your **pom.xml** and notice the version under configuration

```
▼<build>
  ▼<plugins>
    ▼<plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>2.0.2</version>
      ▼<configuration>
        <source>1.4</source>
        <target>1.4</target>
      </configuration>
    </plugin>
  </plugins>
</build>
</project>
```

It is 1.4 since we have created **j2ee 4 project**

Now we will add generic code to this project which can't be compiled with 1.4

Then we will add compiler with configuration **1.5**

To add a code go to `\MyWebApp\src\main\java\org\mywebapp\com` and create one java file there as **MyClass.java**

```
package org.mywebapp.com;

import java.util.*;

public class MyClass
{
    Run | Debug
    public static void main(String[] args)
    {
        List<String> stringList = new ArrayList<String>();
    }
}
```

Now let's try to compile this

In folder till **MyWebApp** path, open **cmd**

mvn compile

We have got error here which clearly indicate we can't compile this java file with version 4,

we need either version 5 or higher

Now to change the version open pom.xml file and edit it as below

```
▼ <build>
  ▼ <plugins>
    ▼ <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>2.0.2</version>
      ▼ <configuration>
        <source>1.5</source>
        <target>1.5</target>
      </configuration>
    </plugin>
  </plugins>
</build>
</project>
```

Now again try to compile the project

In folder till **MyWebApp** path, open **cmd**

mvn compile

Since maven 3 uses compiler 5 by default if we remove the build section from pom.xml and try to clean and compile again it will get build successfully

B. Using server plugin (tomcat or jetty)

We have plugins for tomcat and jetty, these are light weight server containers

For jetty, declare it in pom.xml

```
▼<build>
  ▼<plugins>
    ▼<plugin>
      <groupId>org.eclipse.jetty</groupId>
      <artifactId>jetty-maven-plugin</artifactId>
      <version>9.4.53.v20231009</version>
    </plugin>
  </plugins>
</build>
```

Enter to following command to start with jetty server

mvn jetty:run

Open the browser and type **localhost:8080** (for jetty)

For Tomcat

```
▼<build>
  ▼<plugins>
    ▼<plugin>
      <groupId>org.apache.tomcat.maven</groupId>
      <artifactId>tomcat7-maven-plugin</artifactId>
      <version>2.2</version>
      <!-- Use the appropriate version -->
    </plugin>
  </plugins>
</build>
```

To start with tomcat server

mvn tomcat7:run

Open the browser and type **localhost:8080/yourWebApp**

(for tomcat)

PRACTICAL 11

Creating maven project in eclipse

A. Creating simple Hello World! Web application with maven

1. Open eclipse
2. **Click on file → new → Maven Project**
3. Check the **checkbox** if you want to create a **default project** and don't want to select archetype otherwise keep it unchecked and click on next
4. Enter **webapp-j2ee** in filter select the archetype and click on next
5. Enter the **group id, artifact id, snapshot and package** then click on finish
6. In the console type **Y** for confirmation
7. Maven project is created
8. Right click on your project click on **run as → Maven Install**
9. After it is done check the target folder under your project it should contain **.war file**
10. Again right click on project and select **run as Maven Build...**
11. Enter **goal**
12. Click on **run** and once server started
13. Open browser and enter **localhost:8080/MyWebAppE**

B. Creating simple Calculator Web application with maven

Edit **index.jsp** file as follow:

```
1 <%@page contentType="text/html" pageEncoding="UTF-8"%>
2 <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
3   "http://www.w3.org/TR/html4/loose.dtd">
4
5 <html>
6 <head>
7   <title>Calculator App</title>
8 </head>
9 <body>
10 <form action="CalculatorServlet" method="get" >
11   Enter First Number: <input type="text" name="txtN1" ><br>
12   Enter Second Number: <input type="text" name="txtN2" ><br>
13   Select an Operation
14   <input type="radio" name="opr" value="+">ADDITION
15   <input type="radio" name="opr" value="-">SUBTRACTION
16   <input type="radio" name="opr" value="*">MULTIPLY
17   <input type="radio" name="opr" value="/">DIVIDE <br>
18   <input type="reset">
19   <input type="submit" value="Calculate" >
20 </form>
21 </body>
22 </html>
```

Add a servlet file to the project as **CalculatorServlet.java**

```
1 package org.mywebappE.com.MyWebAppE;
2
3 import java.io.IOException;
4 import java.io.PrintWriter;
5 import javax.servlet.ServletException;
6 import javax.servlet.http.HttpServlet;
7 import javax.servlet.http.HttpServletRequest;
8 import javax.servlet.http.HttpServletResponse;
9 public class CalculatorServlet extends HttpServlet
10 {
11     public void doGet(HttpServletRequest request, HttpServletResponse response)throws ServletException, IOException
12     {
13         response.setContentType("text/html;charset=UTF-8");
14         PrintWriter out = response.getWriter();
15         out.println("<html><head><title>Calculator Servlet</title></head><body>");
16         double n1 = Double.parseDouble(request.getParameter("txtN1"));
17         double n2 = Double.parseDouble(request.getParameter("txtN2"));
18         double result =0;
19         String opr=request.getParameter("opr");
20         if(opr.equals("+"))
21             result=n1+n2;
22         if(opr.equals("-"))
23             result=n1-n2;
24         if(opr.equals("*"))
25             result=n1*n2;
26         if(opr.equals("/"))
27             result=n1/n2;
28         out.println("<h1> Result = "+result);
29         out.println("</body></html>");
30     }
31 }
```

Save the project, build and execute in browser

localhost:8080/MyWebAppE/